ComfyUI 从入门到精通

> **版本与核实信息**
>
> | 项目 | 说明 |
> | --- | --- |
> | 版本基准 | ComfyUI v0.42.x 稳定版本线（2026 年 10 月） |
> | 核实日期 | 2026-10-22 |
> | 主要依据 | 官方文档 docs.comfy.org、GitHub 发布说明（Comfy-Org/ComfyUI） |
> | 环境假设 | 以 Windows 10 及以上、NVIDIA 显卡为主要示例环境；macOS（Apple Silicon）与 Linux 差异在相应位置注明 |
> | 名称约定 | 节点首次出现时同时给出中文界面名称与英文类名，例如「K 采样器（KSampler）」 |
> | 时效性说明 | ComfyUI 迭代较快，个别界面细节、参数默认值请以官方文档 docs.comfy.org 为准 |

第一部分 入门篇：从认识到跑通第一张图

本部分解决「从零到独立完成第一次图像生成」的问题。读者将了解 ComfyUI 的定位与设计思想，完成安装与初始配置，熟悉界面与画布操作，并逐节点理解第一条文生图工作流。

第 1 章 认识 ComfyUI：节点式生成式 AI 应用

初次接触生成式 AI 工具时，常见困惑是：表单式的界面难以精确控制生成过程，想要「先放大、再修脸、最后加背景」这类多步骤处理时无从下手。本章介绍 ComfyUI 如何以节点图的方式解决这一问题，并说明它与同类工具的差别。完成本章学习后，读者能够判断 ComfyUI 是否适合自身需求，并理解后续章节反复出现的核心概念。

1.1 什么是 ComfyUI

ComfyUI 是一个基于节点（Node）的生成式 AI 应用程序与推理引擎，由 comfyanonymous 发起、Comfy Org 维护，完全开源，可在本地设备运行。使用者通过在画布上放置节点、连接连线来组织生成流程，从而生成图像、视频、音频与 3D 内容。

这里有两个需要预先定义的术语：

- **节点（Node）**：执行一项特定任务的模块。有的节点负责加载文件，有的负责修改数据，有的负责生成或保存结果。节点是画布上的方框，方框内有可设置的属性（Property）。
- **工作流（Workflow）**：由连线相连的节点组成的图（Graph），描述「如何一步步生成内容」。从技术上讲，工作流就是一张节点图，这种组织方式也被称为可视化编程（Visual Programming）——通过排列和连接可视化模块来创建程序，而不需要编写代码。

ComfyUI 本身不内置模型权重文件，模型文件需要单独下载并放入指定目录，这一过程将在第 2 章与第 6 章详细说明。

1.2 节点式工作流意味着什么

节点式界面带来的直接价值有三点：

1. **过程透明。**从加载模型到保存图像，每一步都对应一个节点，数据如何流动一目了然。出问题时，报错的节点会被标记出来，便于定位。
2. **控制精确。**每个环节的参数（例如采样步数、放大倍数、控制强度）都暴露在对应节点上，可以单独调整、单独替换。
3. **流程可复用、可分享。**工作流可以保存为 JSON 文件；生成的图像文件中也可以内嵌工作流信息，把图像拖回画布即可恢复完整流程。这种「工作流即文件」的特性是 ComfyUI 生态活跃的重要原因。

代价是上手门槛高于表单式工具：使用者需要理解每个环节的作用，而不是按顺序填写几个输入框。本教程的结构正是为降低这一门槛而设计的。

1.3 与其他工具的对比

下表从使用方式与能力维度对比常见选择。

维度	ComfyUI	WebUI 类工具（以 AUTOMATIC1111、Forge 为代表）	云端生成服务
操作模式	节点图工作流	表单与选项	网页输入框
学习曲线	较陡	平缓	最低
流程灵活性	高，可自由编排多阶段流程	受界面预设限制	受服务功能限制
数据隐私	本地运行，数据不出本机	本地运行	数据上传至服务方
硬件要求	需要本地算力（也提供云端方案）	需要本地算力	无本地算力要求
复用与自动化	工作流文件可分享，提供 API	配置可分享，自动化能力有限	取决于平台

结论性建议：需要多步骤流程、批量自动化或想把生成能力接入自己程序的使用者，适合选择 ComfyUI；只需偶尔生成单张图像的使用者，表单式工具或云端服务可能更省事。

1.4 适用场景

ComfyUI 覆盖的生成领域包括：

- **图像**：文生图、图生图、局部重绘（Inpainting）、扩图（Outpainting）、高清放大；
- **视频**：文生视频、图生视频、运动控制与视频放大；
- **音频**：语音合成、对话生成（部分依赖官方合作伙伴节点，见第 10 章）；
- **3D 与其他模态**：3D 资产生成与预览；
- **工程化场景**：通过 Server API 把工作流嵌入自动化管线（见第 13 章）。

1.5 本教程的学习路径

本教程按「入门篇、进阶篇、精通篇」三部分递进：入门篇完成安装与第一次生成；进阶篇讲透原理、模型体系、提示词与控制技术；精通篇覆盖自定义节点开发、API 集成、性能调优与综合实战。各章独立成篇，读者可按顺序通读，也可在具备基础后跳读。

第 2 章 安装与初始配置

ComfyUI 官方提供桌面版、Windows 便携版与手动安装三条途径，选择困难通常出现在「不清楚三者差别」与「不确定自己的硬件是否达标」。本章给出系统要求、三条安装途径的完整步骤与对比结论，并说明安装后的首次启动与更新方式。完成本章学习后，读者能够根据自身系统与硬件完成安装，并进入可用的操作界面。

2.1 系统要求

安装前先核对硬件与软件条件。

项目	要求
操作系统	Windows 10 及以上；Linux；macOS 13 及以上（仅 Apple Silicon，即 M 系列芯片）
显卡	推荐 NVIDIA 或 AMD 独立显卡；亦支持 Intel Arc、Apple Silicon Metal、CPU 运行（速度慢）
磁盘空间	桌面版每个安装至少 4.85 GB；模型文件另计，单个模型常为数 GB
内存	最低 8 GB，推荐 16 GB
浏览器	推荐 Google Chrome 143 及以上；旧版本存在已知的视觉与性能问题
Python（仅手动安装）	推荐 Python 3.13；3.14 可用但部分自定义节点可能不兼容；3.12 可作备选

说明：无论哪一种安装方式，ComfyUI 都运行在独立的 Python 环境中，不同安装之间互不干扰。Windows 便携版当前内置 Python 3.13 与 PyTorch CUDA 13.0 构建。

2.2 方式一：Comfy Desktop 桌面版

Comfy Desktop 是官方桌面应用，本质是一个多安装管理器：它负责安装、管理与启动多个 ComfyUI 实例，每个实例可拥有独立的版本、自定义节点与设置。桌面版默认跟踪 ComfyUI 的稳定版发布渠道。

安装步骤（以 Windows 为例，macOS 与 Linux 流程类似）：

1. 访问官方下载页 `https://comfy.org/download`，下载对应平台的安装程序（Windows 为 `.exe`，Linux 提供 `.deb` 与 AppImage）。

2. 运行安装程序，按提示完成安装；Windows 上独立安装不需要管理员权限。
3. 从开始菜单或桌面快捷方式启动 Comfy Desktop。
4. 首次启动时，初始设置流程会询问运行方式：选择 **Comfy Cloud**（在云端 GPU 上运行，本机不安装）或 **Local**（在本机安装）。选择 Local 后，安装向导会自动完成 Python 环境与依赖的部署。

桌面版适合大多数用户：一键安装、自动更新，并提供快照与模型下载管理等功能。若需无头（headless）或纯服务器部署，桌面版不支持，应改用便携版或手动安装。

2.3 方式二：Windows 便携版

便携版（Portable）是内置独立 Python 环境（`python_embeded`）的完整 Windows 版本，解压即可使用，适合希望始终使用最新提交、或在无管理员权限环境部署的用户。便携版仅支持 Windows。

步骤：

1. 在官方下载页根据显卡类型选择压缩包：NVIDIA RTX 系列对应 CUDA 13.0 与 Python 3.13 构建；NVIDIA 10 系及更早显卡对应 CUDA 12.6 与 Python 3.12 构建；另有 AMD（ROCm）、Intel 与 Windows on ARM 构建。
2. 使用 7-ZIP 解压压缩包，得到如下目录结构（以 NVIDIA 版为例）：

```
ComfyUI_windows_portable
├── ComfyUI                    # ComfyUI 程序主体
├── python_embeded             # 独立的 Python 环境
├── update                     # 用于升级的批处理脚本
├── README_VERY_IMPORTANT.txt # 便携版使用说明
├── run_cpu.bat                # 双击启动（仅 CPU）
└── run_nvidia_gpu.bat         # 双击启动（NVIDIA 显卡）
```

3. 双击 `run_nvidia_gpu.bat` 启动。命令行窗口出现以下输出即表示服务已启动：

```
To see the GUI go to: http://127.0.0.1:8188
```

4. ComfyUI 通常会自动打开浏览器并访问 `http://127.0.0.1:8188`；若未自动打开，手动访问该地址即可。

注意：使用过程中不能关闭该命令行窗口，否则 ComfyUI 服务将停止运行。如需为便携版补充 Python 包（自定义节点依赖等），使用内置 Python 执行：

```
.\python_embeded\python.exe -m pip install <包名>
```

2.4 方式三：手动安装

手动安装通过 Git 获取源码并自行配置 Python 环境，支持全部系统与 GPU 类型（NVIDIA、AMD、Intel、Apple Silicon、Ascend NPU、Cambricon MLU 等），适合开发者与需要完全控制环境的用户。

以 Windows + NVIDIA 显卡为例，在命令行中依次执行：

```
git clone https://github.com/Comfy-Org/ComfyUI.git
cd ComfyUI
pip install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu130
pip install -r requirements.txt
```

各命令含义：

- `git clone`：将 ComfyUI 源码仓库复制到本地；
- `pip install torch ... --extra-index-url https://download.pytorch.org/whl/cu130`：安装 PyTorch，`cu130` 表示 CUDA 13.0 构建，NVIDIA 20 系及以上显卡要求 cu130 或更高版本；
- `pip install -r requirements.txt`：安装 ComfyUI 声明在 `requirements.txt` 中的全部依赖。

启动服务：

```
python main.py
```

预期输出与便携版相同，出现 `To see the GUI go to: http://127.0.0.1:8188` 即启动成功。macOS（Apple Silicon）用户安装 PyTorch 时使用默认渠道即可获得 Metal 加速支持，其余步骤一致。

2.5 三种方式对比与选择建议

方式	适用系统	版本渠道	上手难度	特点
Comfy Desktop	Windows、macOS（Apple Silicon）、Linux	默认稳定版，可切换	低	多实例管理、自动更新、图形化配置；不支持无头部署
便携版	仅 Windows	最新提交	低	解压即用、免安装、便于移动部署
手动安装	全部	自主控制	中	完全可控，适合开发与服务器部署

建议：普通用户优先选择 Comfy Desktop；需要在固定目录随身携带或使用较新特性时选择便携版；需要无头部署、二次开发或精确控制依赖时选择手动安装。

2.6 更新

- **桌面版**：在实例的更新标签页中选择 Stable 或 Latest on GitHub 渠道进行更新。
- **便携版**：运行 `update` 文件夹内提供的升级批处理脚本（脚本文件名以实际解压内容为准）。
- **手动安装**：在 ComfyUI 目录执行 `git pull`，随后再次执行 `pip install -r requirements.txt` 同步依赖。

第 3 章 界面布局与基本操作

安装完成后，浏览器中打开的画布、节点与侧栏对新手而言信息量较大，「不知道从哪里入手操作」是常见状态。本章拆解界面组成，讲解画布、节点、连线的基本操作与常用快捷键，并说明工作流的保存、加载与模板使用。完成本章学习后，读者应能自如地在画布上完成添加节点、连线、运行与保存等基础操作。

3.1 界面组成

ComfyUI 界面的主要区域：

- **画布**：中央的无限平面，节点与连线在此摆放，是主要工作区。
- **菜单与工具栏**：顶部区域提供工作流（Workflows）菜单（打开、导出、浏览模板等）、运行按钮（Queue 或 Run）与队列管理等。
- **侧栏**：包含节点库（按类别列出可添加的节点）、工作流列表等面板，可通过界面上的图标展开。

关于 **Nodes 2.0**：当前版本的节点渲染已从旧的 LiteGraph Canvas 体系迁移到基于 Vue 的新体系，节点交互更丰富、迭代更快；在极端缩放或超大工作流场景下可能存在少量视觉问题，部分自定义节点需要更新才能完全适配。旧渲染体系仍可在设置中随时切换，二者可自由来回切换。

3.2 画布基本操作

操作	方式
平移画布	按住鼠标中键或空格键拖动（不同平台快捷键可能略有差异，以界面提示为准）
缩放	鼠标滚轮
框选多个节点	在空白处按下左键拖出选框
搜索节点	在画布空白处双击，弹出搜索框
添加节点	双击搜索、右侧栏节点库拖放、或右键菜单添加

3.3 节点操作

- **添加**：双击画布空白处输入节点名称搜索后回车，即可在鼠标位置创建节点。
- **移动与选择**：拖动节点标题栏移动；单击选中，按住 Shift 单击可多选，框选亦可选多个。
- **复制与删除**：选中后使用 `Ctrl+C`、`Ctrl+V` 复制粘贴；按 `Delete` 删除选中节点。
- **属性设置**：节点内部的输入框、下拉列表、滑块即属性（Property），用于控制该节点行为；部分属性也可由其他节点的输出连线提供取值。

3.4 连线规则

连线（Link）从一个节点的输出端口指向另一节点的输入端口，用于在节点间传递数据。核心规则是**只有兼容的数据类型才能连接**，例如图像输出不能直接连接到期望接收数字的输入。

节点端口按数据类型以颜色和标签区分，常见类型如下表。类型名称在后续章节会反复出现，此处只需建立印象，第 4 章与第 5 章将结合工作流具体解释。

类型	常见含义	典型来源节点
MODEL	扩散模型本体	模型加载类节点
CLIP	文本编码器	模型加载类节点
VAE	变分自编码器	模型加载类节点
CONDITIONING	条件编码结果	文本编码类节点
LATENT	潜空间图像数据	空白潜空间、采样器
IMAGE	像素空间图像	解码、加载、保存类节点
数字类（INT/FLOAT/STRING）	整数、浮点数、字符串	各类参数节点

拖拽端口即可建立连线；将已有连线拖到新端口可改接；按住 `Ctrl` 点击连线可断开（以界面实际提示为准）。若两个节点类型兼容但未自动连接，可从输出端口拖到输入端口手动连接。

3.5 常用快捷键

快捷键	作用
`Ctrl + Enter`	将当前工作流加入执行队列（运行）
`R`	刷新节点定义，更新节点下拉列表（例如刚放入新模型文件后）
`Ctrl + O`	打开工作流文件
`Ctrl + S`	保存当前工作流
`Ctrl + C` / `Ctrl + V`	复制、粘贴选中节点
`Delete`	删除选中节点
双击空白处	搜索并添加节点

3.6 工作流的保存、加载与模板

- **保存与加载**：工作流可保存为 JSON 文件并通过 `Workflows → Open`（`Ctrl + O`）重新打开，可独立于生成的媒体进行分享。
- **从图像恢复工作流**：使用内置保存节点输出的图像文件内嵌了工作流元数据，将这类图像直接拖入画布即可恢复其工作流；也可通过 `Workflows → Open` 打开图像。
- **模板库**：通过 `Workflow → Browse Workflow Templates` 打开内置工作流模板。模板提供完整可运行的示例，使用 ComfyUI 自带的核心（Core）节点，并在缺少模型时提示下载，是初学者最可靠的起点。

3.7 运行与队列

点击 Queue 按钮或按 `Ctrl + Enter`，当前工作流即进入执行队列。运行期间可以继续编辑画布、排队多个任务；队列面板中可查看正在执行与排队的任务，并进行取消、清空等操作。若某节点缺失、报错或未收到所需数据，工作流无法完成，该节点会被标记，依据标记定位问题即可。

第 4 章 第一个文生图工作流：核心节点解析

本章目标非常具体：跑通默认文生图工作流，生成第一张图像，并弄清每个节点在做什么。这一工作流是后续所有章节的地基——图生图、重绘、ControlNet 都是在它的基础上增加或替换节点而成。完成本章学习后，读者应能独立完成模型放置、工作流运行，并逐个说出六个核心节点的作用与关键参数。

4.1 准备模型

ComfyUI 默认不带任何模型。本教程使用 Stable Diffusion 1.5 的官方检查点 `v1-5-pruned-emaonly-fp16.safetensors`（约 2 GB），它是后续讲解的标准示例模型。

1. 从 Hugging Face 下载该文件（官方入门文档提供下载链接）。
2. 将文件放入模型目录：

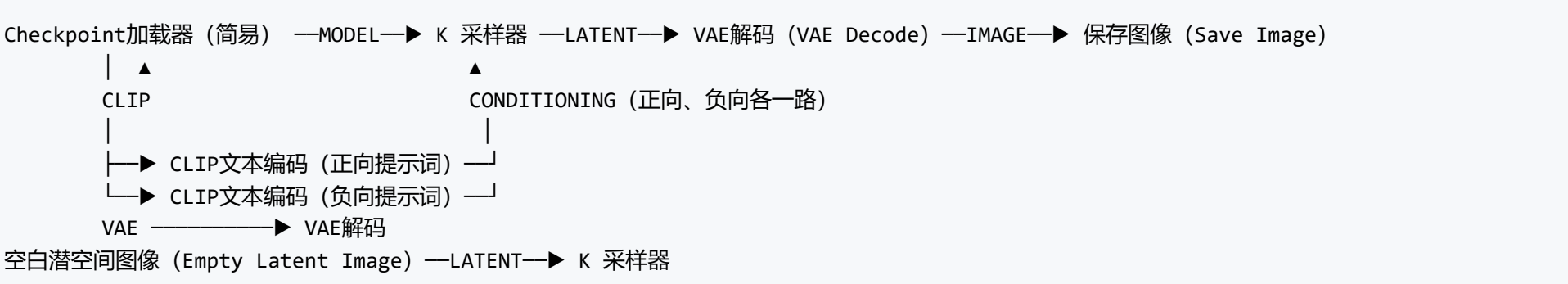
```
ComfyUI/models/checkpoints/
```

目录说明：`ComfyUI/models/` 是模型总目录，其下按类型分子目录，`checkpoints` 存放主模型检查点，其余子目录见第 6 章。

3. 若 ComfyUI 正在运行，在画布上按 `R` 刷新节点定义，使模型出现在下拉列表中；桌面版用户也可通过模板的自动下载提示完成模型获取。

4.2 加载默认工作流

正常启动后，ComfyUI 会自动加载默认文生图工作流；也可以从模板库打开同款模板。该工作流的节点连接关系如下（箭头表示数据流向）：



若觉得连线图抽象，直接对照画布上的节点更直观：每个节点都有明确的中文名称，端口颜色即第 3 章表中的数据类型。

4.3 运行并查看结果

1. 在「Checkpoint加载器（简易）」节点的下拉列表中确认已选中 `v1-5-pruned-emaonly-fp16.safetensors`。
2. 点击 Queue 按钮或按 `Ctrl + Enter` 运行。
3. 运行完成后，结果出现在「保存图像」节点中，右键该图像即可保存到本地；图像文件同时保存在 `ComfyUI/output/` 目录。

若对结果不满意，可直接再次运行：K 采样器每次会依据 `seed` 参数生成不同的随机种子，因此每次结果不同。控制随机性的方法见 7.5 节。

4.4 核心节点逐个解析

Checkpoint加载器（CheckpointLoaderSimple，界面名「Checkpoint加载器（简易）」）：加载主模型文件。检查点（Checkpoint）通常打包了三个组成部分，对应节点的三个输出端口：

- **MODEL（UNet）**：扩散模型主干，负责在采样过程中预测并去除噪声、生成图像；
- **CLIP**：文本编码器，把提示词文本转换为模型可理解的向量，模型无法直接理解文字；
- **VAE**：变分自编码器（Variational AutoEncoder），负责在像素空间与潜空间之间转换图像。

CLIP文本编码（CLIPTextEncode）：把提示词编码为条件（Conditioning）。工作流中有两个实例：输出连接到 K 采样器 `positive` 端口的是正向提示词（希望出现的元素），连接到 `negative` 端口的是负向提示词（不希望出现的元素）。

空白潜空间图像（EmptyLatentImage）：构造一张纯噪声的潜空间图像，其宽高属性决定了最终生成图像的尺寸，相当于设定画布大小。潜空间（Latent Space）的概念见第 5 章，此处可理解为「尺寸减半再减半的压缩画布」。

K 采样器（KSampler）：整个工作流的核心，去噪生成过程在此发生，输出潜空间图像。参数含义如下表，调优方法见第 7 章。

参数	含义	作用
model	用于去噪的扩散模型	决定生成图像的风格与质量
positive	正向提示词条件编码	引导生成包含指定元素
negative	负向提示词条件编码	抑制不希望出现的内容
latent_image	待去噪的潜空间图像	噪声初始化的输入载体
seed	随机种子	控制生成随机性，同一种子同参数结果一致
control_after_generate	生成后的种子控制方式	决定批量生成时种子的变化规律
steps	去噪迭代步数	步数越多细节越充分，耗时越长
cfg	无分类器引导系数（Classifier-Free Guidance）	控制提示词约束强度，过高会导致画面过拟合、色彩过饱和
sampler_name	采样算法名称	决定去噪路径的数学方法
scheduler	调度器类型	控制噪声衰减速率与步长分配
denoise	去噪强度系数	0.0 保留输入原图，1.0 完全重绘

VAE解码（VAEDecode）：把采样输出的潜空间图像解码回像素空间，得到可查看的图像。

保存图像（SaveImage）：把图像保存到 `ComfyUI/output/`，默认文件名前缀为 `ComfyUI`，并内嵌工作流元数据。

4.5 完整数据流回顾

把六个节点串起来，一次生成的过程是：

1. 检查点加载器输出 MODEL、CLIP、VAE 三路数据；
2. 两路提示词经 CLIP 编码为正、负条件；
3. 空白潜空间图像按设定尺寸构造噪声；
4. K 采样器以种子初始化噪声，在正负条件引导下按 steps 步去噪，输出潜空间图像；
5. VAE 解码将潜空间图像转换回像素图像；
6. 保存图像节点写盘。

4.6 动手修改

建议依次尝试以下修改并观察变化，每项修改后按 `Ctrl + Enter` 重新运行：

- 修改正向提示词的描述内容（英文书写、逗号分隔的写法规范见第 7 章）；
- 将 `Empty Latent Image` 的宽高从 512 调整为 640；
- 固定 `seed` 数值后再次运行，观察同一种子下结果的一致性；
- 将 `steps` 从 20 调整为 10 与 30，对比细节与耗时。

第二部分 进阶篇：系统掌握生成与控制

本部分解决「知道怎么点按钮，但不知道为什么、如何控制效果」的问题。读者将理解扩散模型与工作流的运行机制，掌握模型体系的组织方式，学会用提示词与采样参数控制质量，并掌握图生图、局部重绘、放大与 ControlNet 等关键控制技术。

第 5 章 扩散模型与工作流运行原理

上一章的工作流能够生成图像，但「为什么噪声能变成图像」「潜空间到底是什么」不解决，后续的参数调优与故障判断就只能是死记硬背。本章讲清扩散模型的基本原理、潜空间与 VAE 的关系、条件的引导机制，以及 ComfyUI 执行工作流的方式。完成本章学习后，读者应能从原理层面解释第 4 章中每个参数的作用。

5.1 扩散模型的基本原理

扩散模型（Diffusion Model）是一类通过「加噪—去噪」过程学习图像分布的生成模型。训练阶段，模型学习对图像逐步添加高斯噪声，并学习逆转这一过程；生成阶段则完全在反向进行：从一张纯高斯噪声出发，模型反复预测并去除其中的噪声，经过若干步迭代后得到一张符合训练分布的清晰图像。

第 4 章使用的 `v1-5-pruned-emaonly-fp16.safetensors` 就是这样一个训练完成的模型：只需要输入提示词作为条件，它就能通过对随机噪声逐步去噪生成目标图像。K 采样器的 `steps` 参数即去噪迭代的步数，`denoise` 为 1.0 时表示从纯噪声开始完整去噪。

5.2 潜空间与像素空间

潜空间（Latent Space）是扩散模型使用的抽象数据表示：图像先被压缩为尺寸更小、信息更稠密的潜表示，再在潜空间中执行去噪。以 SD1.5 为例，压缩比例为 8 倍，一张 512×512 的图像对应 64×64×4 的潜表示，计算量显著降低。

像素空间（Pixel Space）即最终看到的图像本身所在的存储空间。两个空间之间的转换由 **VAE（变分自编码器，Variational AutoEncoder）**完成：编码（Encode）把像素图像压缩为潜表示，解码（Decode）把潜表示还原为像素图像。这解释了第 4 章工作流末尾必须有「VAE 解码」节点——采样在潜空间进行，而人眼观看的是像素空间。潜空间分辨率的整除约束也来源于此：宽高需要是 8 的倍数。

5.3 条件引导与 CLIP

模型无法直接理解文字。**CLIP** 是文本编码器，把提示词转换为语义向量，作为去噪过程的**条件（Conditioning）**注入每一步去噪。正向条件把去噪方向「拉向」提示词描述的内容，负向条件把方向「推离」负向提示词描述的内容。

引导强度由 `cfg`（无分类器引导系数，Classifier-Free Guidance）控制：数值越高，画面越服从提示词约束；过高会导致过拟合，表现为色彩过饱和、画面僵硬。SD1.5 类模型的常用范围约为 7 至 8，新一代模型往往使用更低的引导值（如 FLUX 类模型使用 `guidance` 参数），具体以所用模型的官方工作流为准。

5.4 采样器与调度器

- **采样器（Sampler，对应 `sampler_name`）**：决定去噪路径的具体数学方法，例如 `euler`（欧拉法）是一阶的稳定基础方法。
- **调度器（Scheduler，对应 `scheduler`）**：决定各步之间噪声衰减的安排方式，例如 `normal` 为线性安排，`karras` 在尾部安排更细的步长。

建议以 `euler` + `normal` 作为理解基准组合，需要更精细细节时尝试 `dpmpp_2m` + `karras` 等常用组合。不同采样器在不同步数下表现差异较大，比较时务必固定其他参数。

5.5 ComfyUI 如何执行工作流

理解执行机制对排查问题与优化速度都有帮助：

- **执行顺序**：ComfyUI 按依赖关系对节点图排序后执行，从数据源头逐级推进到输出节点。执行顺序可能因缓存状态等因素变化，不应依赖固定的执行顺序。
- **结果缓存**：同一会话中，输入未发生变化的节点会复用上次的结果，只有受修改影响的节点会重新执行。这正是「只改提示词再运行时速度较快」的原因。调试时如需每次完整重算，可使用 `--cache-none` 启动参数（见第 14 章）。
- **错误定位**：节点报错时该节点被标记，同时控制台输出详细日志，二者结合即可定位问题。

5.6 检查点内部结构与新一代模型

传统检查点（SD1.5、SDXL）把扩散主干（UNet）、文本编码器（CLIP）与 VAE 打包在一个文件中，因此一个「Checkpoint加载器」节点即可输出三路数据。新一代模型（FLUX、Qwen-Image 等）通常将扩散主干、文本编码器、VAE 分开提供，分别放入 `diffusion_models`、`clip`、`vae` 目录，并由多个独立加载节点组合，工作流形态与 SD1.5 不同。识别方法与加载方式见第 6 章。

第 6 章 模型体系：Checkpoint、LoRA、VAE 与模型管理

模型文件动辄数 GB，类型繁多，放错目录或选错加载节点是新手最常见的失败点之一。本章建立一套完整的模型管理框架：模型类型与目录的对应关系、下载渠道、各类加载节点的用法、LoRA 的叠加方法，以及通过配置文件共享模型目录的方法。完成本章学习后，读者应能为任意官方模板正确部署其所需模型。

6.1 模型文件与格式

模型（Model）在本教程语境中指包含网络训练结果的权重文件，例如扩散检查点、CLIP 文本编码器、放大模型等。文件格式以 `.safetensors` 为主，它是安全的张量存储格式，不携带可执行代码，优先于旧格式 `.ckpt` 使用。GGUF 量化格式的模型不受原生支持，需要安装社区自定义节点 ComfyUI-GGUF 后使用。

6.2 模型类型与目录对应关系

ComfyUI 启动时扫描 `ComfyUI/models/` 下的类型子目录来发现模型。常用对应关系如下：

模型类型	存放目录	典型加载节点	用途
检查点（Checkpoint）	`models/checkpoints/`	Checkpoint加载器（简易）	包含扩散主干、CLIP、VAE 的完整生图模型
LoRA	`models/loras/`	LoRA加载器（Load LoRA）	轻量附加权重，用于风格、角色、概念定制
VAE	`models/vae/`	VAE加载器（VAE Loader）	潜空间与像素空间的转换器
ControlNet	`models/controlnet/`	ControlNet模型加载器（Load ControlNet Model）	结构控制条件模型
放大模型	`models/upscale_models/`	使用模型放大（Upscale Image using Model）	超分辨率放大（如 RealESRGAN 系列）
嵌入（Embedding）	`models/embeddings/`	在提示词中按文件名引用	文本反演词向量
文本编码器	`models/clip/`	CLIP加载器（Load Clip）	新一代模型的独立文本编码器
扩散主干	`models/diffusion_models/`	加载扩散模型（UNET Loader）	新一代模型的独立扩散主干

注意：单独的 VAE 文件仅当模型自带的 VAE 不可用或效果不佳时才需要，多数检查点已内置 VAE。

6.3 下载渠道

- **Hugging Face**（huggingface.co）：官方模型发布的主渠道，第 4 章的 SD1.5 检查点即由此获取。
- **Civitai**（civitai.green）：社区模型聚合站，检查点、LoRA 丰富，页面通常标注基础模型版本与触发词。
- **GitHub 项目页**：部分模型与配套工作流一同发布。

下载时注意模型的基础架构（SD1.5、SDXL、FLUX 等）须与工作流匹配，文件体积通常为数 GB，需预留磁盘空间与下载时间。

6.4 加载节点的组合逻辑

识别一个模型该用什么节点，依据是其文件形态：

- 单文件包含全部组件 → 「Checkpoint加载器（简易）」；
- 仅扩散主干 → 「加载扩散模型（UNET Loader）」，再分别用 CLIP 加载器、VAE 加载器补齐另两个组件；
- 附加权重 → 「LoRA加载器」串接在模型通路上。

放入文件后按 `R` 刷新节点定义即可在下拉列表中看到；若仍不可见，重启 ComfyUI。模型删除没有卸载按钮，直接从对应目录删除文件即可，删除前建议确认不再被工作流引用。

6.5 LoRA 的原理与叠加

LoRA（Low-Rank Adaptation，低秩适配）是在基础模型之外训练的小体积附加权重文件，通常几十 MB 至几百 MB，用于为模型注入特定风格、角色或概念。其使用方式是在工作流中将「LoRA加载器」节点插入模型通路：

```
Checkpoint加载器 ──MODEL──▶ LoRA加载器 ──MODEL──▶ K 采样器
        └─CLIP──▶ LoRA加载器 ──CLIP──▶ CLIP文本编码
```

「LoRA加载器」有 `strength`（模型强度）与 `strength_clip`（文本编码器强度）两个常用参数，取值以 1.0 为标称强度，需要更克制的效果时降低取值。叠加多个 LoRA 时，将多个加载器依次串联，各节点强度分别控制；叠加过多或强度过高可能出现风格冲突与画面劣化。部分 LoRA 依赖**触发词（Trigger Word）**——训练时约定的特定提示词，需写入正向提示词才会生效，具体以模型发布页说明为准。

6.6 常见模型版本线与显存压力

版本线	典型生成分辨率	特点
SD1.5	512×512	体积小、显存要求低、生态最成熟
SDXL	1024×1024 附近	细节与构图能力更强，显存需求更高
SD3.5 / FLUX / Qwen-Image 等	1024×1024 及以上	组件分离存放，提示词遵循能力强，显存需求最高

显存不足时的应对手段（量化模型、启动参数等）见第 14 章。

6.7 用 extra_model_paths.yaml 共享模型目录

多个 ComfyUI 实例、或与其他工具（如 WebUI）共用模型时，可通过配置文件添加额外模型搜索路径，避免重复占用磁盘。便携版与手动安装在 ComfyUI 根目录将 `extra_model_paths.yaml.example` 复制并改名为 `extra_model_paths.yaml` 后编辑；桌面版在应用的 Storage 标签页中添加，对应文件位于 `C:\Users\<用户名>\AppData\Roaming\ComfyUI\extra_models_config.yaml`（Windows）。

配置示例——假设模型集中存放在 `D:\models` 下：

```yaml
my_models:
    base_path: D:\models
    checkpoints: checkpoints/
    loras: loras/
    vae: vae/
    controlnet: controlnet/
    upscale_models: upscale_models/
```

保存后重启 ComfyUI 生效；启动日志会显示已加载的额外路径，可用于确认配置是否生效。

6.8 模型找不到的排查顺序

1. 确认文件位于正确的类型子目录；
2. 按 `R` 刷新节点定义；
3. 重启 ComfyUI；
4. 检查加载节点下拉列表中是否选中了正确文件；
5. 若使用额外路径配置，检查启动日志中路径是否加载成功。

第 7 章 提示词工程与采样参数调优

同一个工作流，不同人用出的效果差距很大，差别通常集中在提示词写法与 K 采样器参数两组因素上。本章给出一套可操作的提示词写法规则与参数调优基准，使生成质量可控、可复现。完成本章学习后，读者应能针对 SD1.5 类模型写出结构清晰的提示词，并按目标调整采样参数。

7.1 提示词的基本写法

针对 SD1.5 类模型，官方教程给出的基础规则：

- 尽量使用英文；
- 用英文逗号分隔描述片段；
- 使用短语而非完整长句；
- 描述具体化，例如指定光线、视角、材质等具体属性；
- 可使用 `masterpiece, best quality` 等质量类关键词改善出图质量。

正向提示词描述希望出现的内容；负向提示词描述希望回避的内容，例如常见的 `bad hands`（第 4 章默认工作流即以此为例）。

7.2 权重语法

提示词中可用 `(关键词:数值)` 语法调整某片段的权重，数值即权重倍率：

```
(golden hour:1.2)
```

该写法将 `golden hour`（黄金时刻光线）的权重提高为 1.2 倍，使其更倾向于出现在画面中。权重数值建议从接近 1.0 的幅度开始微调，过大容易出现画面劣化。注意：权重语法是 SD1.5 系生态的习惯写法，新一代模型对自然语言的遵循能力强，通常无需依赖权重微调。

7.3 新一代模型的提示词风格

FLUX、Qwen-Image 等新一代模型以自然语言长句描述见长，官方模板示例多为完整的场景描述句，而非逗号分隔的关键词堆叠。迁移建议：对这类模型先用一句完整的英文描述（主体、动作、环境、光线、风格），效果不足再做局部补充。

7.4 采样参数调优基准

以第 4 章默认工作流为基准的调优建议：

目标	调整手段	说明
提升细节	提高 `steps`（如 20 → 30）	耗时线性增加，收益递减
结果可复现	固定 `seed`，`control_after_generate` 设为 fixed	同种子同参数结果一致，便于对比
批量探索	`control_after_generate` 设为 increase 等	每次运行种子按规律变化
画面更服从提示词	微调 `cfg`	SD1.5 常用 7 至 8，过高易过饱和
改变采样特性	更换 `sampler_name` 与 `scheduler` 组合	比较时固定其余参数

7.5 分辨率与批处理

`Empty Latent Image` 节点控制生成分辨率与批量：

- 宽高须为 8 的倍数（潜空间 8 倍压缩的整除约束，见 5.2 节）；
- 建议以所用模型的原生训练分辨率为中心调整（SD1.5 为 512 级，SDXL 为 1024 级），偏离过大会放大构图缺陷；
- `batch_size` 属性一次生成多张，适合配合固定参数扫种子。

7.6 调优工作法

建议遵循「单变量」原则：每次只修改一个参数并固定 `seed` 对比，避免多变量同时变化导致无法归因。将满意的参数组合保存进工作流文件（JSON 中包含全部节点属性），即可作为个人基准工作流复用。

第 8 章 图生图、局部重绘与高清放大

仅有「从噪声生成」一种模式无法覆盖日常需求：改照片风格、修局部、把小图变大图是高频任务。本章在文生图工作流的基础上讲解三条扩展路线：图生图（以图起始）、遮罩重绘（区域级修改）与放大（分辨率提升）。完成本章学习后，读者应能独立完成风格迁移、局部修改与两阶段高清修复。

8.1 图生图（Image to Image）

图生图把「纯噪声起点」替换为「真实图像加噪后的潜表示」，其余流程不变：

```
加载图像（Load Image）──IMAGE──▶ VAE编码（VAE Encode）──LATENT──▶ K 采样器（denoise < 1.0）
```

关键参数是 K 采样器的 `denoise`：0.0 完全保留输入、1.0 完全重绘（等同于文生图）。常用中间值权衡「保留原图程度」与「改变幅度」：0.4 至 0.6 偏保留，0.7 以上偏重绘。注意：图生图工作流的提示词仍然有效，描述的是改图的目标方向。

8.2 遮罩与局部重绘（Inpainting）

局部重绘只重绘画面指定区域。核心是三步：

1. **提供遮罩**：在「加载图像」节点上右键选择在遮罩编辑器（MaskEditor）中打开，用画笔涂抹要修改的区域；或使用「加载图像遮罩（Load Image Mask）」节点读取带透明通道的图像。
2. **编码潜表示**：使用「VAE编码（用于重绘，VAE Encode for Inpainting）」节点替代普通 VAE 编码，该节点在编码时处理遮罩区域，为重绘提供合适的起始状态。
3. **采样**：K 采样器 `denoise` 保持接近 1.0（遮罩区域内完全重绘），采样结果仅在遮罩区域被替换。

实践中可接入「遮罩成长（Grow Mask）」类节点将遮罩范围外扩数像素，使重绘区域与原图过渡更自然。若重绘边缘出现接缝，优先检查遮罩是否覆盖完整目标区域并适当外扩。

8.3 扩图（Outpainting）

扩图向画面外扩展内容。核心节点为「为扩图填充图像（Pad Image for Outpainting）」：它按设定的上下左右像素扩展画布并自动生成待填充区域的遮罩，之后走与 8.2 相同的重绘采样流程。扩展幅度较大时建议分多次小步扩图，逐步向四周延展，避免大范围一次性生成导致边缘不协调。

8.4 高清放大

放大有两条路线，实践中常组合使用：

- **潜空间放大再采样（高清修复）**：对 K 采样器输出的潜表示接「上采样潜空间（Upscale Latent By）」节点放大（常用倍率 1.5 至 2.0），再接第二个 K 采样器以较低 `denoise`（约 0.3 至 0.5）二次采样，补充高分辨率下的细节。该路线在放大的同时修正细节，是经

典的两阶段高清修复结构。

- **模型放大（超分辨率）**：对解码后的像素图像接「使用模型放大（Upscale Image using Model）」节点，配合 `models/upscale_models/` 中的超分模型（如 RealESRGAN 系列），另有「放大图像（Upscale Image）」节点做普通插值缩放。该路线速度快、不改变内容语义，但不产生新细节。

推荐顺序：先潜空间两阶段生成大图，再接超分模型做最终尺寸放大。放大模型文件放入 `models/upscale_models/` 后按 `R` 刷新即可选用。

8.5 人像细节修复

面部、手部等细节的修复手段按侵入程度从低到高：

1. 对问题区域做局部重绘（见 8.2 节）；
2. 两阶段高清修复（见 8.4 节）整体提升细节密度；
3. 专用修复节点：面部修复、换脸等专项功能多由社区自定义节点提供，安装与管理见第 11 章。

8.6 工作流整理：节点组与子图

流程变长后，整理能力开始影响效率：

- **成组**：框选相关节点后创建组框（Group）并命名，标注功能分区；
- **子图（Subgraph）**：较新版本支持把选中节点转换为子图节点，对外只暴露关键输入输出，可显著压缩画布复杂度，官方模板中已有大量使用；
- **复用**：把调好的阶段（如「高清修复段」）另存为独立工作流文件，按需导入。

第 9 章 ControlNet 与精确控制

提示词只能描述「要什么」，控制不了「构图如何、姿态怎样」；让生成结果严格贴合参考图的轮廓、深度或人体姿态，需要 ControlNet。本章讲解 ControlNet 的工作原理、节点接法、参数含义与常用控制类型，并介绍角色一致性的相关技术。完成本章学习后，读者应能用姿态图或边缘图精确控制生成结果。

9.1 ControlNet 是什么

ControlNet 是一类附加条件模型：它在扩散模型之外，把一张**控制图**（例如人体姿态图、深度图、边缘图）编码为额外条件注入去噪过程，使生成结果的结构与控制图对齐。提示词继续负责「内容与风格」，ControlNet 负责「结构与构图」，二者叠加工作。

ControlNet 模型与基础模型版本必须配套（SD1.5 模型配 SD1.5 的 ControlNet，SDXL 同理），文件放入 `models/controlnet/`。

9.2 节点接法

标准接法在文生图工作流的条件通路上插入三个节点：

```
参考图（Load Image）──IMAGE──▶ 预处理节点 ──IMAGE──▶ ControlNet应用（高级）
                                                          ├──▶ positive / negative ──▶ K 采样器
ControlNet模型加载器 ──CONTROL_NET──▶ ControlNet应用（高级）
```

各节点职责：

- **ControlNet模型加载器（Load ControlNet Model）**：加载 `models/controlnet/` 中的模型文件；
- **预处理节点**：从参考图提取控制图。核心节点提供 Canny（边缘检测）等基础预处理器，更多预处理器（姿态、深度等）通过社区预处理节点包获得（见第 11 章）；
- **ControlNet应用（高级，Apply ControlNet (Advanced)）**：把控制图条件分别叠加到正、负两条条件通路上，输出仍为 CONDITIONING 类型，接回 K 采样器。

9.3 关键参数

「ControlNet应用（高级）」节点的三个参数：

参数	含义	调整建议
strength	控制强度	1.0 为完全服从控制图；构图想更自由时降至 0.6 至 0.8
start_percent	开始生效的采样进度（0 至 1）	设为大于 0 可让前期采样更自由
end_percent	停止生效的采样进度	设为小于 1 可只在早期约束构图，后期由提示词补充细节

9.4 常用控制类型

控制类型	控制图来源	适用场景
Canny	边缘检测	保持物体轮廓与线稿结构
Depth	深度图	保持空间层次与前后关系
OpenPose	人体姿态骨架	精确控制人物姿态动作
MLSD	直线段检测	建筑、室内等强透视场景
Normal	法线贴图	表面起伏与光照结构
Seg	语义分割	按区域指定物体类别与布局

选择原则：需要「结构完全一致」选 Canny 或 Depth；需要「只约束人物姿态」选 OpenPose；需要按区域控制场景元素布局选 Seg。

9.5 多 ControlNet 叠加

复杂控制可同时叠加多个 ControlNet，例如 OpenPose 控制姿态加 Depth 控制空间：串联两个「ControlNet应用（高级）」节点，各自接入对应模型与控制图，条件逐级传递。叠加时建议各自降低 `strength`（例如各 0.7 左右），避免条件冲突导致画面混乱。

9.6 角色一致性与参考生成

让同一角色跨图保持外观一致，当前有三条技术路线：

1. **LoRA 训练**：为目标角色训练专属 LoRA（见第 6 章），配合触发词调用，一致性最好，但需要离线训练步骤；
2. **参考注入类节点**：IPAdapter 等社区自定义节点把参考图特征注入生成过程，无需训练；
3. **原生参考生成**：较新的核心节点（如 v0.41.0 引入的 ReferenceMultiLatent）与新一代模型的原生参考图能力，可直接以参考图引导生成，具体用法以官方模板为准。

建议按「先试原生参考与参考注入，不满意再训练 LoRA」的顺序投入成本。

第三部分 精通篇：扩展、实战与工程化

本部分解决「从个人使用走向熟练与工程化」的问题：扩展到视频、音频等多模态生成，驾驭自定义节点生态，开发自己的节点，通过 API 集成自动化，优化性能并系统排查故障，最终以综合项目收束全篇。

第 10 章 视频、音频与多模态生成

图像工作流掌握之后，视频、语音与 3D 是自然的扩展方向；这些领域模型更新快、参数多，直接动手容易迷失。本章给出多模态生成的整体框架：视频工作流与图像工作流的差异、音频生成的主要形态、3D 与合作伙伴节点的概念，并以官方模板作为可靠入口。完成本章学习后，读者应能依据模板独立跑通一条图生视频或语音生成流程。

10.1 视频生成工作流的形态

视频生成在 ComfyUI 中的两种基本形态：

- **文生视频（Text to Video）**：从文本直接生成视频片段；
- **图生视频（Image to Video）**：以一张图像为起始帧，生成运动视频，是目前可控性最好、使用最广的形态。

当前主流开源视频模型包括 Wan 系列、LTX 系列（如 LTX-2.5）、HunyuanVideo 等，官方持续在更新说明与版本记录中新增支持。模型迭代较快，具体可用模型与推荐参数以官方模板库与版本记录为准。

10.2 视频工作流与图像工作流的差异

在节点层面，视频工作流与图像工作流的主要差异：

- **潜表示带有时间维度**：视频模型的潜表示在图像的空间维度之外增加了帧维度，采样在「帧序列」上进行；
- **参数增加**：在分辨率之外引入帧数（或时长）、帧率等参数，帧数与分辨率共同决定显存占用，视频生成的显存需求显著高于图像生成；
- **解码与保存节点不同**：VAE 解码输出的是图像序列，最终由「保存视频（Save Video）」类节点封装为视频文件（节点内置编码参数，如 H.264 的 CRF 质量值）。

显存不足时优先降低分辨率或帧数，再考虑量化模型与启动参数（第 14 章）。

10.3 音频生成

ComfyUI 核心提供基础音频节点（如加载音频 Load Audio、预览音频、保存音频），支持读取、处理与输出音频。完整的语音合成与对话生成能力当前主要通过**合作伙伴节点（Partner Nodes）**提供，例如 ElevenLabs 的语音与对话模型、Fish Audio 的对话模型、HeyGen Voice 的即时语音克隆与文本转语音等。

合作伙伴节点是官方聚合的付费 API 节点：使用前需在界面中登录账号，调用时请求远端服务并按用量计费，不需要本地下载模型。这一形态与本地模型形成互补：本地模型免费但占显存，合作伙伴节点无本地算力要求但产生费用。若不希望加载此类节点，可使用 `--disable-partner-nodes` 启动参数；`--offline` 参数则会进一步限制前端的一切外网访问。

10.4 3D 与其他模态

ComfyUI 亦提供 3D 内容支持：核心包含 3D 加载与预览类节点（如 Load 3D），官方模板中有 3D 相关工作流分类。3D 工作流同样遵循「节点图」范式，本教程不再展开，使用者可按模板与官方文档按需学习。

10.5 以模板为入口的多模态学习路径

多模态生成的推荐学习路径：

1. 打开 `Workflow → Browse Workflow Templates`，按图像生成视频、音频、3D 分类浏览；
2. 选择目标模板运行，缺少模型时按提示自动下载；
3. 对照模板逐节点理解参数含义，方法与第 4 章相同；
4. 修改帧数、分辨率、提示词等参数观察变化，逐步改造成自己的流程。

官方学习页（comfy.org/learning）提供按领域分类的免费视频教程，覆盖节点图基础、文生图与图生图、LoRA 与 ControlNet、重绘放大、图生视频与运动控制等主题，可作为本教程的配套材料。

第 11 章 自定义节点生态与管理

核心节点只覆盖通用能力，姿态预处理、批量管理、特定模型加载等大量实用功能由社区自定义节点提供；同时，安装来源不明的节点也存在依赖冲突与安全风险。本章讲解自定义节点的安装、管理与排查方法，并给出风险控制建议。完成本章学习后，读者应能安全地扩充节点库，并独立解决「节点缺失」这一最高频问题。

11.1 自定义节点是什么

自定义节点（Custom Node）是以 Python 包形式安装的节点集合，通常托管在 GitHub。安装后，节点出现在画布与节点库中，与核心节点用法一致。安装位置统一为：

```
ComfyUI/custom_nodes/<节点包目录>/
```

11.2 ComfyUI-Manager：节点管理中枢

ComfyUI-Manager 是官方维护的节点管理扩展，新版本 ComfyUI 已内置：通过界面上的插件图标或菜单中的「管理扩展」入口打开，提供以下核心功能：

- 搜索并安装社区节点包；
- 更新、禁用、启用、卸载已安装的节点包；
- **安装缺失节点（Install Missing Custom Nodes）**：打开一个来自他人的工作流后，若画布出现红色缺失节点，该功能可自动识别并一键补装对应的节点包。

若在管理器中找不到界面，可检查设置中的 Manager 显示选项（新版管理器界面默认启用，旧版界面可在设置中切换）。

11.3 手动安装

不经过管理器的安装方式：

1. **Git 克隆**：在 `custom_nodes` 目录执行 `git clone <仓库地址>`；
2. **ZIP 解压**：下载仓库 ZIP，解压到 `custom_nodes` 下，确保目录内包含 `__init__.py`；
3. 安装节点包声明的 Python 依赖：多数节点包在 README 中说明，通常在其目录内执行 `pip install -r requirements.txt`（便携版使用 `.\python_embeded\python.exe -m pip install -r ComfyUI\custom_nodes\<包名>\requirements.txt`）；
4. 重启 ComfyUI，节点出现在库中即安装成功。

11.4 风险控制与依赖冲突

自定义节点直接运行第三方代码，且各包的 Python 依赖可能相互冲突。建议遵循以下原则：

- 优先选择活跃维护、使用者多的节点包，安装前阅读其 README 与更新记录；
- 每次只安装一个新节点包并重启验证，便于定位冲突来源；
- 安装来源不明的节点包前，确认代码仓库可信；
- 重要环境操作前备份：至少保存好 `models`、`custom_nodes` 与个人工作流文件；桌面版提供快照（Snapshot）功能可用于恢复；
- 排查依赖类故障时可整体卸载近期安装的节点包再逐个回装。

11.5 常用节点包示例

节点包	功能	典型用途
ComfyUI-Manager	节点管理中枢	安装、更新、缺失节点补装
rgthree-comfy	流程辅助与整理类节点	提升画布组织效率
ComfyUI-GGUF	GGUF 量化模型加载	低显存运行大模型
controlnet_aux 类预处理包	ControlNet 预处理器	姿态、深度等控制图提取

以上仅为示例，选择节点包时应以「当前任务需要什么能力」为准，避免盲目堆积。

11.6 缺失节点与异常排查

- **节点显示红色**：表示该节点缺失，对应节点包未安装。依次尝试：用管理器的「安装缺失节点」补装；按 `R` 刷新；重启。
- **节点加载报错**：查看启动日志中该包的错误输出，多为依赖缺失或版本不兼容，按其 README 重新安装依赖。
- **怀疑某节点包导致异常**：使用 `--disable-all-custom-nodes` 启动参数跳过全部自定义节点验证问题归属，再用 `--whitelist-custom-nodes <目录名>` 放行指定节点包二分定位。

11.7 更新与兼容

ComfyUI 与自定义节点需要协同更新：核心版本升级后，个别节点包可能需要更新才能适配（界面渲染体系迁移至 Nodes 2.0 的过程中即有此现象）。官方提供了**节点替换（Node Replacement）**机制：开发者可为其已废弃的节点注册到新节点的迁移映射，前端检测到工作流中的旧节点时会提示一键升级，并自动映射输入输出。使用者只需在提示出现时确认升级，连线与参数会被保留。

第 12 章 自定义节点开发（V3 schema）

当现有节点都无法满足需求时，编写自己的节点是最终手段。本章以官方推荐的 V3 schema 为准，讲解节点包结构、最小可用节点的完整代码、与旧版 V1 写法的对应关系，以及节点替换与发布方法。完成本章学习后，读者应能编写并调试一个自定义节点包。

12.1 开发环境与包结构

一个自定义节点包就是一个 Python 目录，最小结构如下：

```
my_custom_nodes/
├── __init__.py            # 入口：定义 comfy_entrypoint()
├── nodes.py               # 节点实现
└── node_replacements.py   # （可选）节点替换注册
```

将该目录放入 `ComfyUI/custom_nodes/`，重启 ComfyUI 即被加载。开发调试时修改代码后重启生效；控制台日志会输出加载过程中的报错，是调试的第一手信息。

12.2 V3 schema：最小可用节点

以下代码定义了一个最简单的节点：接收一个字符串，返回追加后缀的结果。可直接作为起点改造。

```python
from comfy_api.latest import ComfyExtension, io


class MySuffixNode(io.ComfyNode):
    @classmethod
    def define_schema(cls) -> io.Schema:
        return io.Schema(
            node_id="MySuffixNode",
            display_name="My Suffix Node",
            category="my_category",
            inputs=[
                io.String.Input("text", default="hello"),
            ],
            outputs=[io.String.Output()],
        )

    @classmethod
    def execute(cls, text) -> io.NodeOutput:
        return io.NodeOutput(text + "_suffix")


class MyExtension(ComfyExtension):
    async def get_node_list(self) -> list[type[io.ComfyNode]]:
        return [MySuffixNode]


async def comfy_entrypoint() -> ComfyExtension:
    return MyExtension()
```

代码要点：

- `from comfy_api.latest import ...` 引用当前最新版本的 Comfy API；也可以引用具体版本号（如 `comfy_api.v0_0_2`）以获得更稳定的接口；
- `define_schema()` 用 `io.Schema` 声明节点的 `node_id`（全局唯一标识，自定义节点应加前后缀避免与他人的类名冲突）、`display_name`（界面显示名）、`category`（节点库分类路径）、`inputs` 与 `outputs`；
- 执行方法固定命名为 `execute`，且为类方法；返回 `io.NodeOutput`；
- `comfy_entrypoint()` 是包的入口函数，返回包含节点列表的 `ComfyExtension` 对象；
- V3 中节点对象不保存实例状态，`__init__` 中设置的实例属性不会反映到节点行为。

进度反馈等进阶能力（如执行中预览图像）由 API 提供，此处不展开，完整接口见官方自定义节点文档。

12.3 与 V1 写法的对应关系

历史上自定义节点以 V1 风格编写，目前 V1 写法仍然可用。V1 与 V3 的关键差异：

方面	V1（旧）	V3（现行）
输入输出定义	`INPUT_TYPES()` 返回字典	`define_schema()` 中的 Schema 对象
执行方法	类内任意命名，由 `FUNCTION` 指定	固定为类方法 `execute`
暴露节点	`NODE_CLASS_MAPPINGS` 等模块级字典	`comfy_entrypoint()` 返回扩展对象
状态	依赖实例属性	类方法，无实例状态
新特性	不再获得新增能力	后续功能只加入 V3

V1 示例（等价于 12.2 节的功能）：

```python
class MySuffixNodeV1:
    @classmethod
    def INPUT_TYPES(s):
        return {"required": {"text": ("STRING", {"default": "hello"})}}

    RETURN_TYPES = ("STRING",)
    FUNCTION = "run"
    CATEGORY = "my_category"

    def run(self, text):
        return (text + "_suffix",)


NODE_CLASS_MAPPINGS = {"MySuffixNodeV1": MySuffixNodeV1}
```

新开发的节点建议直接采用 V3；维护旧节点时可在理解映射关系后逐步迁移。

12.4 调试建议

- 每次修改代码后重启 ComfyUI 再测试；
- 关注启动日志中本节点包的导入错误与注册信息；
- 先用最小输入输出验证骨架，再逐步增加参数；
- 节点执行出错时，错误堆栈会打印在控制台，依据堆栈定位到具体行。

12.5 节点替换（Node Replacement）

节点重命名、合并或输入参数变更时，注册替换映射可让旧工作流自动迁移。在包内创建 `node_replacements.py`，并在扩展的 `on_load` 生命周期中注册：

```python
from comfy_api.latest import ComfyAPI, io

api = ComfyAPI()


class MyExtension(io.ComfyExtension):
    async def on_load(self) -> None:
        await api.node_replacement.register(io.NodeReplace(
            new_node_id="MyNewNode",
            old_node_id="MyOldNode",
            old_widget_ids=["steps", "cfg"],
            input_mapping=[
                {"new_id": "model", "old_id": "model"},
                {"new_id": "num_steps", "old_id": "steps"},
            ],
            output_mapping=[
                {"new_idx": 0, "old_idx": 0},
            ],
        ))
```

要点：`old_widget_ids` 列出旧节点按位置存储的控件参数名，因为工作流 JSON 按位置索引保存控件取值，该列表使替换系统能正确迁移参数；`input_mapping` 建立新旧参数的对应，也可用 `set_value` 为新参数设定固定值。注册后，前端会从服务端获取替换表，检测到旧节点时提示用户升级并自动应用映射。

12.6 发布

发布流程：在 GitHub 创建仓库并推送代码，编写 README 说明功能、安装方式、依赖与示例工作流；使用者即可通过 ComfyUI-Manager 搜索安装，或通过 Git 克隆安装。建议在 README 中注明适配的 ComfyUI 版本线，降低兼容性问题。

第 13 章 API 调用与自动化

ComfyUI 的价值不止于交互式画布：它同时是一个可通过 HTTP 与 WebSocket 访问的生成服务，适合把工作流嵌入批量处理、网页应用或自动化管线。本章讲解 Server API 的核心用法：工作流 API 格式的导出、提交与查询、WebSocket 监听，以及局域网部署与安全注意事项。完成本章学习后，读者应能用 Python 脚本驱动一条完整工作流并取回生成结果。

13.1 Server API 概览

ComfyUI 启动后在 `http://127.0.0.1:8188` 提供 REST 风格的 HTTP 接口与 WebSocket 通道。本章使用的端点：

端点	方法	用途
`/prompt`	POST	提交工作流（API 格式 JSON）加入执行队列，返回 `prompt_id`
`/history/{prompt_id}`	GET	查询某次执行的输出记录
`/view`	GET	按文件名下载输出文件
`/ws`	WebSocket	实时接收执行状态与预览数据
`/queue`	GET	查看当前队列
`/object_info`	GET	查询全部节点定义与参数规格

依赖说明：本章示例仅使用 Python 标准库与 `websocket-client` 包（`pip install websocket-client`）。

13.2 导出工作流的 API 格式

画布上的工作流文件与 API 调用所需的格式并不相同：API 格式以「节点编号为键」组织，每个节点只保留 `class_type` 与 `inputs`。导出方法：在 `Workflows` 菜单中选择导出为 API 格式（Export (API)），得到 JSON 文件供脚本加载与改写。

API 格式的结构示例（节选自默认文生图工作流）：

```
{
  "3": {
    "class_type": "KSampler",
    "inputs": {
      "seed": 8566257, "steps": 20, "cfg": 8,
      "sampler_name": "euler", "scheduler": "normal", "denoise": 1,
      "model": ["4", 0], "positive": ["6", 0], "negative": ["7", 0],
      "latent_image": ["5", 0]
    }
  },
  "4": {
    "class_type": "CheckpointLoaderSimple",
    "inputs": {"ckpt_name": "v1-5-pruned-emaonly.safetensors"}
  }
}
```

其中 `["4", 0]` 表示「取第 4 号节点的第 0 个输出端口」，即画布连线的 JSON 表示。

13.3 提交与取回结果

完整示例：提交默认文生图工作流，经 WebSocket 等待执行完成，再从 `/history` 取记录、经 `/view` 下载图像。该模式是官方推荐的通用模式。

```python
import json
import urllib.request
import urllib.parse
import uuid
import websocket  # pip install websocket-client

SERVER = "127.0.0.1:8188"
client_id = str(uuid.uuid4())


def queue_prompt(prompt):
    payload = json.dumps({"prompt": prompt, "client_id": client_id}).encode("utf-8")
    req = urllib.request.Request(f"http://{SERVER}/prompt", data=payload)
    return json.loads(urllib.request.urlopen(req).read())["prompt_id"]


def get_image(filename, subfolder, folder_type):
    params = urllib.parse.urlencode({
        "filename": filename, "subfolder": subfolder, "type": folder_type,
    })
    with urllib.request.urlopen(f"http://{SERVER}/view?{params}") as resp:
        return resp.read()


def get_history(prompt_id):
    with urllib.request.urlopen(f"http://{SERVER}/history/{prompt_id}") as resp:
        return json.loads(resp.read())


def get_images(ws, prompt):
    prompt_id = queue_prompt(prompt)
    while True:                          # 等待执行完成
        message = json.loads(ws.recv())
        if message["type"] == "executing":
            data = message["data"]
            if data["node"] is None and data["prompt_id"] == prompt_id:
                break                    # node 为 None 表示本次执行结束
    outputs = get_history(prompt_id)[prompt_id]["outputs"]
    images = []
    for node_output in outputs.values():
        for image in node_output.get("images", []):
            images.append(get_image(image["filename"], image["subfolder"], image["type"]))
    return images


if __name__ == "__main__":
    with open("workflow_api.json", encoding="utf-8") as f:
        prompt = json.load(f)
    prompt["3"]["inputs"]["seed"] = 5          # 按节点编号改写参数
    prompt["6"]["inputs"]["text"] = "masterpiece best quality man"

    ws = websocket.WebSocket()
    ws.connect(f"ws://{SERVER}/ws?clientId={client_id}")
    for data in get_images(ws, prompt):
        print(f"received {len(data)} bytes")
    ws.close()
```

预期输出：脚本打印若干行 `received <字节数> bytes`，表示图像已按字节取回，可写入文件或交给后续处理；同时 `ComfyUI/output/` 中会出现对应文件。二进制帧中还包含执行过程中的预览图，如需实时预览可解析 WebSocket 二进制帧（格式见官方 Server Messages 文档）；若希望在取图阶段不经磁盘，可在工作流中改用内置的 SaveImageWebsocket 节点，由 WebSocket 直接推送图像数据。

13.4 批量自动化要点

把单次调用扩展为批量任务时的要点：

- 参数化改写：循环中修改提示词、种子、尺寸等字段后重复提交；
- 队列并发：`/prompt` 会自动排队，无需自行管理并发；
- 失败处理：依据 `/history` 是否返回该 `prompt_id` 的记录判断完成与否，异常时记录节点与错误信息；
- 工作流版本管理：将 `workflow_api.json` 纳入版本控制，与画布工作流文件同步更新。

13.5 局域网部署与安全

默认情况下服务仅监听本机（`127.0.0.1`）。跨设备访问时调整监听地址与端口：

```
python main.py --listen 0.0.0.0 --port 8188
```

- `--listen`：监听地址，`0.0.0.0` 表示所有网络接口；不带值时同样默认监听全部接口；
- `--port`：监听端口，默认 8188；
- `--tls-keyfile` 与 `--tls-certfile`：配置后启用 HTTPS。

安全提示：ComfyUI 默认没有身份验证机制，任何能访问该端口的人都可以提交任务并读取文件。直接暴露到公网存在数据泄露与资源滥用风险。防护措施：仅在可信局域网内开放监听；必须公网提供时，在前置反向代理层增加鉴权与访问控制，并启用 HTTPS。

第 14 章 性能优化与故障排除

生成速度慢、显存不足、输出黑图、启动报错，是使用中最高频的四类问题。本章以官方启动参数为工具箱，讲解性能优化的系统方法与常见故障的定位流程。完成本章学习后，读者应能针对自己的硬件选择合理的启动参数，并按流程独立排除常见故障。

14.1 显存管理机制

ComfyUI 默认自动管理模型在显存与内存之间的调度（在支持的硬件上启用动态显存管理，并在 NVIDIA 与 AMD 显卡上默认启用异步权重卸载）。一般情况无需手动干预；仅在默认策略不合适时，用以下互斥参数覆盖默认行为：

参数	行为
`--gpu-only`	全部模型与计算放在 GPU
`--highvram`	模型常驻显存，使用后不卸载
`--lowvram`	文本编码器等在 CPU 上运行以降低显存占用
`--novram`	显存占用压到最低，速度进一步下降
`--cpu`	全部使用 CPU，极慢但无显存要求
`--reserve-vram <GB>`	为操作系统保留指定 GB 的显存

判断建议：生成报显存不足时，优先尝试 `--lowvram`，仍不足用 `--novram`；系统卡顿时用 `--reserve-vram` 留出余量。

14.2 精度与量化

降低精度是减少显存占用的另一条路线，相关参数按作用对象分组：

- 全局：`--force-fp16`、`--force-fp32`；
- 扩散主干：`--fp8_e4m3fn-unet` 等 fp8 存储（e4m3fn 布局适合 NVIDIA Ada 架构及更新的显卡）；
- 文本编码器：`--fp8_e4m3fn-text-enc` 等；
- VAE：`--fp16-vae`、`--fp32-vae` 等。

注意事项：`--fp16-vae` 在部分环境下可能输出全黑图像，出现黑图时可改用 `--fp32-vae` 验证。除启动参数外，直接使用官方发布的 fp8 量化版模型文件是更常用的省显存方式。

14.3 注意力与实验性优化

注意力实现影响速度与显存，通常保持默认即可；需要干预时可使用 `--use-pytorch-cross-attention`、`--use-sage-attention`、`--use-flash-attention`、`--disable-xformers` 等。实验性提速参数 `--fast` 会启用可能影响质量或稳定性的优化，适合在可接受质量变化的前提下追求速度，单个优化也可单独启用（如 `--fast fp16_accumulation`）。

其他常规优化按收益排序：降低生成分辨率、减少 `steps`、使用量化模型、关闭不必要的预览（`--preview-method` 支持none、auto、latent2rgb、taesd）。调优时保持第 7 章的单变量原则。

14.4 缓存相关参数

第 5 章已说明节点结果缓存机制。相关参数：

- `--cache-none`：关闭缓存，每次运行全部重算，速度慢，适合调试与排除「缓存导致的假象」；
- `--cache-lru <N>`：限制缓存节点结果数量，控制内存占用；

- `--high-ram`：内存充裕时略微改善性能。

14.5 常见故障与处理

现象	可能原因	处理步骤
提示 CUDA out of memory	显存不足	降低分辨率或帧数；量化模型；`--lowvram`、`--novram`；`--reserve-vram` 为系统留余量
模型在下拉列表找不到	目录错误或列表未刷新	核对目录类型；按 `R`；重启；检查额外路径配置与启动日志
节点显示红色	节点包未安装	管理器中安装缺失节点；`R` 刷新；重启
输出全黑图像	VAE 半精度兼容问题	改用 `--fp32-vae`；更换配套 VAE 文件
启动即报错退出	依赖损坏或驱动过旧	查看控制台错误输出；更新显卡驱动；重装依赖
自定义节点互相冲突	依赖版本冲突	逐个禁用定位（`--disable-all-custom-nodes` 配合白名单二分）
界面视觉异常	浏览器版本过旧	更新至 Chrome 143 及以上；必要时切换 Nodes 2.0 渲染开关
重复运行并未加速	缓存被关闭	检查是否误用了 `--cache-none`

14.6 日志与求助路径

排障信息集中在两处：启动 ComfyUI 的命令行窗口（服务端日志，含节点加载与执行报错）与浏览器界面（节点级错误标记）。需要更详细日志时使用 `--verbose` 参数。自助与求助路径依次为：官方文档（docs.comfy.org，含安装、概念、教程与开发章节）、官方论坛与 GitHub 仓库 Issues、社区教程与视频课程。提交问题前，建议整理出可复现步骤、ComfyUI 版本号、启动参数与控制台报错原文，可显著提高获得有效回复的概率。

第 15 章 综合实战项目

前十四章分别解决了单点问题，实战的价值在于把节点、模型、控制与自动化串成完整流程。本章提供三个由易到难的实战项目与一套工作流复用规范，作为从「会用」到「用精」的验收。完成本章学习后，读者应能独立完成从需求分析、工作流搭建到批量交付的完整过程。

15.1 项目一：电商产品主图

需求：为一件商品生成纯色背景的展示图，并输出高清版本。

实现路径：

1. 以文生图工作流（第 4 章）为基底，用一句话正向提示词描述商品与布光，负向提示词排除杂物与文字水印；
2. 若需要贴合指定构图，接入 ControlNet（第 9 章）：用 Canny 控制轮廓，`strength` 取 0.7 至 0.9；
3. 两阶段高清修复（第 8 章）：潜空间放大 1.5 至 2.0 倍后二次采样，最后接超分模型放大到交付尺寸；
4. 固定 `seed`，用 `batch_size` 扫多个种子择优。

验收标准：输出 3 至 5 张候选图，至少 1 张构图符合要求且分辨率达到交付标准。

15.2 项目二：角色一致性插画集

需求：让同一角色出现在不同场景的 8 张插画中。

实现路径：

1. 按第 9 章的路线选择一致性方案：先用原生参考生成或参考注入节点快速验证，效果不满足再训练角色 LoRA；
2. 建立角色描述卡：把角色外貌写成一段稳定复用的提示词模板，配合触发词（如使用 LoRA）；
3. 场景变化：仅替换环境与动作描述段，固定角色描述段与 LoRA 强度；
4. 用「子图」把「角色注入段」封装为一个节点（第 8 章），场景侧自由扩展；
5. 逐张用局部重绘修正手部与面部。

验收标准：8 张图中角色外貌特征保持一致，无需人工比对即可识别为同一角色。

15.3 项目三：短视频素材自动化管线

需求：从一批产品图自动生成图生视频片段，输出统一规格的视频文件。

实现路径：

1. 从模板库选择图生视频模板（第 10 章），跑通单条流程并记录所需模型；
2. 用第 13 章的脚本模式：导出工作流 API 格式，编写批量脚本，循环改写起始图像与提示词后提交；
3. 在脚本中固定输出参数（分辨率、帧数、编码质量），保证规格统一；
4. 用 `/history` 与 `/view` 收集结果，异常任务记录重试；
5. 服务端启用 `--listen` 供局域网内的提交机访问，并按第 13 章的安全建议控制访问范围。

验收标准：批量任务无人值守完成，失败任务可从日志定位原因。

15.4 工作流复用规范

长期使用中，工作流资产的规范程度决定复用效率：

- **命名与分组**：按功能分区创建组框并命名，关键节点使用有意义的标题；
- **参数外置**：把需要频繁修改的参数（提示词、种子、尺寸）集中放在入口节点或子图暴露面，避免在深层节点中翻找；
- **版本存档**：重要工作流以 JSON 文件版本化管理，每次大改另存新版本；
- **分享格式**：对外分享时附带生成示例图（内嵌工作流元数据）与模型清单（模型名、来源、放置目录），接收方按清单部署后即可运行。

15.5 持续精进路径

ComfyUI 生态的演进速度较快，建议建立三个习惯：

1. **跟踪官方版本记录**：docs.comfy.org 的 Changelog 页汇总每次更新的新模型支持、模板与接口变化，是判断「要不要升级」的第一手依据；
2. **以模板库为学习源**：官方模板随版本更新，新模型出现后通常伴随可运行的模板，从模板反推节点接法是最高效的学习方式；
3. **沉淀个人工作流库**：将本章的复用规范长期执行，形成属于自己的、按任务分类的工作流资产库。

至此，本教程覆盖了从安装配置、原理理解、生成控制，到扩展开发、自动化集成与性能优化的完整路径。后续遇到本教程未覆盖的具体模型与节点，按「先查模板、再查官方文档、最后查社区」的顺序即可自行解决。